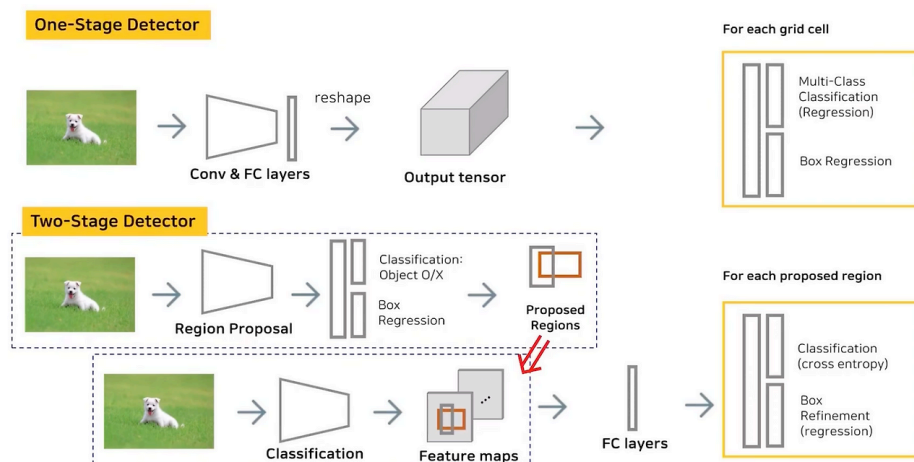


YOLO 논문리뷰

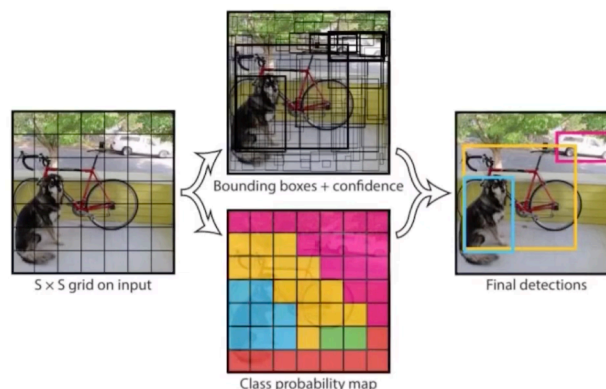
YOLO이전의 R-CNN은 이미지 안에서 bounding box를 생성하기 위해 sliding window를 사용하여 region proposal 방식으로 사진의 부분부분을 확인한다. bounding box에 classifier를 적용하여 분류한 뒤 bounding box를 조정하고, 중복된 검출을 제거하고, 객체에 따라 box의 점수를 재산정하기 위해 후처리를 한다. 반면 YOLO는 이미지 전체를 한눈에 보고 regression으로 multi task를 한번에 처리한다. 이렇게 간단히만 알아봐도 처리속도가 훨씬 빠르고 실시간 처리에 적합할 것이 쉽게 파악된다.

Object Detection에는 두 가지 방식이 존재한다. Image Classification과 Localization을 구분 짓는 Two-stage Detector와 두 과정을 구분 짓지 않는 One-stage Detector가 있다. Two-stage Detector는 성능이 뛰어나지만, 속도가 느리다는 단점이 있다. 대표적인 모델로 Faster R-CNN이 있는데, 실행 속도가 5FPS인 반면 YOLO는 One-Stage Detector로 그 속도가 45FPS(Fast-YOLO는 150FPS)로 매우 빠르다. 두 방식의 차이는 아래 그림에서 한눈에 볼 수 있다.



Unified Detection

YOLO는 region proposal, feature extraction, classification, box regression 과정들을 one-stage detection에 끝내는 것이 핵심이다. 해당 과정은 아래 그림과 같이 작동한다.



우선 Input 이미지를 S x S 크기의 Grid로 나눈다. 한 Grid Cell 안에 어떤 객체의 중심이 포함되면, 해당 Grid Cell이 그 객체에 대해 책임을 지고 탐지를 수행한다.

각 Grid Cell은 B 개의 Bounding Box와 Confidence Score를 예측한다. Confidence Score는 아래와 같이 계산된다.

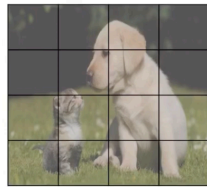
$$\Pr(\text{Object}) * \text{IOU}_{\text{pred}}^{\text{truth}}$$

Confidence Score는 박스 안에 객체가 포함될 확률과, 박스가 객체를 얼마나 잘 담고 있는지에 대한 곱으로 표현된다. $\Pr(\text{Object})$ 는 객체가 포함되면 1, 없으면 0. 그래서 객체가 포함되지 않은 경우 전체가 0이 된다. 점수가 높기 위해서는 예측한 Bounding Box가 최대한 Ground truth box와 겹치도록 하여 1에 가까워야 한다.

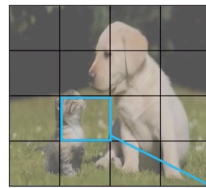
Bounding Box는 4개의 값에 대한 예측으로 이루어져 있다. x, y 는 Bounding Box의 중심 값으로 Grid Cell 안에서의 상대적인 좌표로 0에서 1 사이의 값을 갖게 된다. 예를 들어, bounding Box와 Grid Cell의 중심이 겹치면 x, y 는 0.5, 0.5가 된다. w, h 는 Bounding Box의 너비, 높이 값으로 전체 이미지 안에서 상대적인 길이가 된다. 그래서 최소 0부터 최대 1 값을 가질 수 있다. 예를 들어 Bounding Box의 크기가 이미지의 전체 크기와 같다면, w, h 모두 1이 된다.

Unified Detection

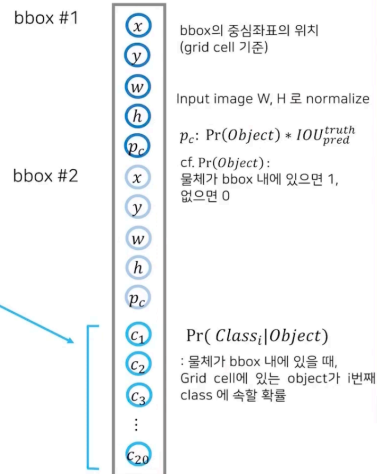
예시) $S = 4, B = 2, C = 20$



Resized image 를
4x4 grid 로 분할



Grid cell 마다 bbox 2개씩 예측



각 Grid Cell은 C 개 클래스에 대한 Conditional Class Probabilities를 계산한다. 각 Bounding Box 안에 있는 객체가 어떤 클래스일지에 대한 조건부 확률이다. Conditional Class Probabilities는 다음과 같이 계산된다.

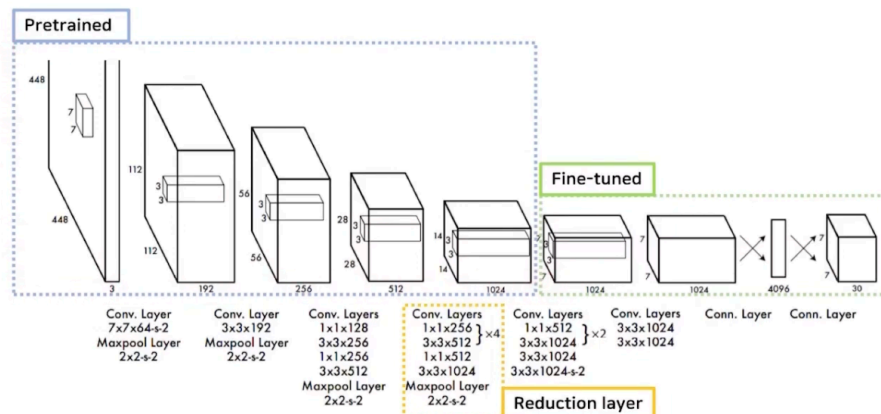
$$\Pr(\text{Class}_i | \text{Object})$$

그리고 Conditional Class Probabilities를 사용해서 최종적으로 구하는 값은 Class-specific Confidence Score이다. Class-specific Confidence Score는 Bounding Box안에 특정 객체가 속할 확률과 Bounding Box가 객체를 얼마나 잘 담고 있는지에 대한 정보를 표현하는 값이며, 다음과 같이 계산된다.

$$\Pr(\text{Class}_i | \text{Object}) * \Pr(\text{Object}) * \text{IOU}_{\text{pred}}^{\text{truth}} = \Pr(\text{Class}_i) * \text{IOU}_{\text{pred}}^{\text{truth}}$$

Architecture

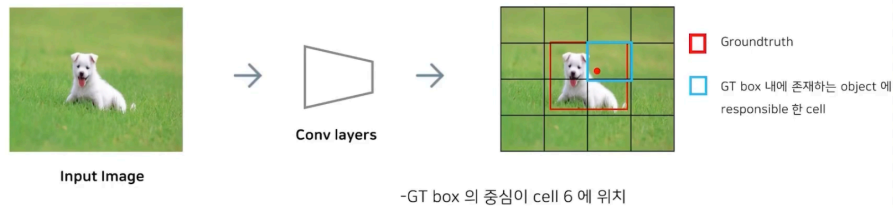
YOLO의 기본 아키텍처는 GoogleLeNet을 차용했다.



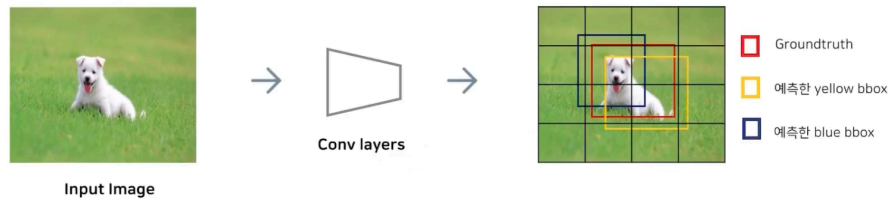
YOLO는 총 24개의 컨볼루션 계층(convolutional layers)과 2개의 전결합 계층(fully connected layers)으로 구성되어 있다. GoogLeNet의 인셉션 구조 대신 YOLO는 1 x 1 축소 계층(reduction layer)과 3 x 3 컨볼루션 계층의 결합을 사용했다. 1 x 1 축소 계층(reduction layer)과 3 x 3 컨볼루션 계층의 결합이 인셉션 구조를 대신한다. 이 모델의 최종 아웃풋은 7 x 7 x 30의 prediction tensors이다. YOLO는 총 24개의 컨볼루션 계층(convolutional layers)과 2개의 전결합 계층(fully connected layers)으로 구성되어 있다. GoogLeNet의 인셉션 구조 대신 YOLO는 1 x 1 축소 계층(reduction layer)과 3 x 3 컨볼루션 계층의 결합을 사용했다. 1 x 1 축소 계층(reduction layer)과 3 x 3 컨볼루션 계층의 결합이 인셉션 구조를 대신한다. 이 모델의 최종 아웃풋은 7 x 7 x 30의 prediction tensors이다.

Training

특정 object에 responsible한 cell i는 GT box의 중심이 위치하는 cell 로 할당



Yolo는 여러 bbox를 예측하지만, 학습단계에서는 IOU(truth pred)가 가장 높은 bbox 1개만 사용한다. 1(obj ij)라는 스칼라 값으로 cell i번째에서 responsible 한 j번째 bbox 를 표시하여 loss function 에 반영한다.



예를 들어 위의 경우 예측한 bbox는 두개지만 GT와 가장 비스한 Yellow bbox만 사용하는 것이다.

$$\begin{aligned}
 & \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} \left[(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2 \right] \\
 & + \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} \left[\left(\sqrt{w_i} - \sqrt{\hat{w}_i} \right)^2 + \left(\sqrt{h_i} - \sqrt{\hat{h}_i} \right)^2 \right] \\
 & + \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} (C_i - \hat{C}_i)^2 \\
 & + \lambda_{\text{noobj}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{noobj}} (C_i - \hat{C}_i)^2 \\
 & + \sum_{i=0}^{S^2} \mathbb{1}_i^{\text{obj}} \sum_{c \in \text{classes}} (p_i(c) - \hat{p}_i(c))^2
 \end{aligned}$$

위 식은 학습에 사용되는 Loss function이다.

빨간색으로 표시한 위의 두줄은 모든 grid cell에서 예측한 B개의 bbox좌표와 GT box 좌표의 오차를 구하는 부분이고, 파란색으로 표시한 다음 두줄은 모든 grid cell에서 예측한 B개의 $\Pr(\text{Class}|\text{Object})$ 와 GT 값의 오차를 구하는 부분이고, 마지막 초록색으로 표시한 한줄은 모든 grid cell의 $\Pr(\text{Object}) * \text{IOU}(\text{truth pred})$ 예측값과 GT box 값의 오차를 구하는 부분이다.

- $\mathbb{1}_{ij}^{obj}$: if jth bbox predictor in cell i is responsible for prediction
- $\mathbb{1}_i^{obj}$: object appears in cell i
- $\lambda_{coord} = 5$: bbox coordinates loss 반영 ↑
- $\lambda_{noobj} = 0.5$: no object 의 class probability loss 반영 ↓

결론적으로 grid cell에 object 존재하는 경우의 오차와 predictor box로 선정된 경우의 오차만 학습한다는 것이다.

Inference

훈련 단계와 마찬가지로, 추론 단계에서도 테스트 이미지로부터 객체를 검출하는 데에는 하나의 신경망 계산만 하면 된다. 파스칼 VOC 데이터 셋에 대해서 YOLO는 한 이미지 당 98개의 bounding box를 예측해주고, 그 bounding box마다 클래스 확률(class probabilities)을 구해준다. YOLO는 테스트 단계에서 굉장히 빠르다. 왜냐하면 YOLO는 R-CNN 등과 다르게 하나의 신경망 계산(a single network evaluation)만 필요하기 때문이다. 하지만 YOLO의 그리드 디자인(grid design)은 한 가지 단점이 있다. 하나의 객체를 여러 그리드 셀이 동시에 검출하는 경우가 있다는 점이다. 객체의 크기가 크거나 객체가 그리드 셀 경계에 인접해 있는 경우, 그 객체에 대한 bounding box가 여러 개 생길 수 있다. 즉, 하나의 그리드 셀이 아닌 여러 그리드 셀에서 해당 객체에 대한 bounding box를 예측할 수 있다는 뜻이다. 하나의 객체가 정확히 하나의 그리드 셀에만 존재하는 경우에는 이런 문제가 없지만 객체의 크기, 객체의 위치에 따라 충분히 이런 문제가 발생할 수 있다. 이를 다중 검출(multiple detections) 문제라고 한다. 이런 다중 검출(multiple detections) 문제는 NMS(non-maximal suppression)라는 방법을 통해 개선할 수 있다. YOLO는 NMS를 통해 mAP을 2~3%가량 향상시킨다.

